

LAPORAN TUGAS

STRUKTUR DATA

Disusun Oleh:

Haykal Muhammad Dahnil

2511532014

Dosen Pengampu :	Dr. Wahyudi, S.T, M.T.
Asisten Praktikum :	Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

1.1 Class PetaMall_2511532014

1.1.1 Kode Program

```
1 package pekan9_2511532014;
2
3 import java.awt.*;
4 import java.util.*;
5 import javax.swing.*;
6
7 public class PetaMall_2511532014 extends JFrame {
8
9     /**
10      *
11      */
12     private static final long serialVersionUID = 1L;
13     private HashMap<String, ArrayList<String>> graph_2014;
14     private HashMap<String, Point> posisi_2014;
15
16     private JComboBox<String> cmbStart_2014;
17     private JComboBox<String> cmbGoal_2014;
18
19     private JTextArea hasil_2014;
20     private JPanel panelGraph_2014;
21
22     private ArrayList<String> jalur_2014 = new ArrayList<>();
23     private ArrayList<String> kunjungan_2014 = new ArrayList<>();
24
25     public PetaMall_2511532014() {
26         setTitle("BFS dan DFS Mall - 2511532014");
27         setSize(1200, 750);
28         setDefaultCloseOperation(EXIT_ON_CLOSE);
29         setLocationRelativeTo(null);
30
31         inisialisasiGraph_2014();
32
33         setLayout(new BorderLayout());
34
35         panelGraph_2014 = new JPanel() {
36             protected void paintComponent(Graphics g) {
37                 super.paintComponent(g);
38                 displayGraph_2014(g);
39             }
40         };
41
42         JPanel kontrol_2014 = new JPanel();
43
44         cmbStart_2014 = new
45             JComboBox<>(graph_2014.keySet().toArray(new String[0]));
46         cmbGoal_2014 = new
47             JComboBox<>(graph_2014.keySet().toArray(new String[0]));
48
49         JButton btnBFS_2014 = new JButton("BFS");
50         JButton btnDFS_2014 = new JButton("DFS");
51         JButton btnReset_2014 = new JButton("Reset");
52
53         kontrol_2014.add(new JLabel("Start"));
54         kontrol_2014.add(cmbStart_2014);
```

```

53     kontrol_2014.add(new JLabel("Goal"));
54     kontrol_2014.add(cmbGoal_2014);
55     kontrol_2014.add(btnBFS_2014);
56     kontrol_2014.add(btnDFS_2014);
57     kontrol_2014.add(btnReset_2014);
58
59     hasil_2014 = new JTextArea();
60     hasil_2014.setEditable(false);
61
62     btnBFS_2014.addActionListener(e -> BFS_2014());
63     btnDFS_2014.addActionListener(e -> DFS_2014());
64     btnReset_2014.addActionListener(e -> resetGraph_2014());
65
66     add(kontrol_2014, BorderLayout.NORTH);
67     add(panelGraph_2014, BorderLayout.CENTER);
68     add(new JScrollPane(hasil_2014), BorderLayout.SOUTH);
69 }
70
71 private void inisialisasiGraph_2014() {
72     graph_2014 = new HashMap<>();
73     posisi_2014 = new HashMap<>();
74
75     String[] node_2014 = {
76         "PintuMasuk", "Lobi", "FoodCourt",
77         "Bioskop", "Supermarket", "TokoBaju",
78         "TokoSepatu", "ATM", "Toilet",
79         "Lift", "AreaBermain"
80     };
81
82     for (String s : node_2014) {
83         graph_2014.put(s, new ArrayList<>());
84     }
85
86     tambahEdge_2014("PintuMasuk", "Lobi");
87     tambahEdge_2014("Lobi", "FoodCourt");
88     tambahEdge_2014("Lobi", "Supermarket");
89     tambahEdge_2014("Lobi", "Lift");
90     tambahEdge_2014("FoodCourt", "Bioskop");
91     tambahEdge_2014("FoodCourt", "Toilet");
92     tambahEdge_2014("Supermarket", "ATM");
93     tambahEdge_2014("Supermarket", "TokoBaju");
94     tambahEdge_2014("TokoBaju", "TokoSepatu");
95     tambahEdge_2014("TokoSepatu", "AreaBermain");
96     tambahEdge_2014("Lift", "Bioskop");
97     tambahEdge_2014("Lift", "AreaBermain");
98     tambahEdge_2014("ATM", "Toilet");
99     tambahEdge_2014("ATM", "Lift");
100    tambahEdge_2014("Toilet", "AreaBermain");
101
102
103    posisi_2014.put("PintuMasuk", new Point(80, 340));
104
105    posisi_2014.put("Lobi", new Point(240, 340));
106
107    posisi_2014.put("FoodCourt", new Point(430, 140));
108    posisi_2014.put("Lift", new Point(430, 340));
109    posisi_2014.put("Supermarket", new Point(430, 540));

```

```

110
111     posisi_2014.put("Bioskop",      new Point(650, 80));
112     posisi_2014.put("Toilet",       new Point(650, 240));
113     posisi_2014.put("ATM",          new Point(650, 440));
114     posisi_2014.put("TokoBaju",     new Point(650, 600));
115
116     posisi_2014.put("AreaBermain", new Point(880, 180));
117     posisi_2014.put("TokoSepatu",   new Point(880, 520));
118 }
119
120 private void tambahEdge_2014(String a_2014, String b_2014) {
121     graph_2014.get(a_2014).add(b_2014);
122     graph_2014.get(b_2014).add(a_2014);
123 }
124
125 private void BFS_2014() {
126     resetData_2014();
127
128     String start_2014 =
129     cmbStart_2014.getSelectedItem().toString();
130     String goal_2014 =
131     cmbGoal_2014.getSelectedItem().toString();
132
133     Queue<String> queue_2014 = new LinkedList<>();
134     HashSet<String> visited_2014 = new HashSet<>();
135     HashMap<String, String> parent_2014 = new HashMap<>();
136
137     queue_2014.offer(start_2014);
138     visited_2014.add(start_2014);
139
140     while (!queue_2014.isEmpty()) {
141         String current_2014 = queue_2014.poll();
142
143         kunjungan_2014.add(current_2014);
144
145         if (current_2014.equals(goal_2014))
146             break;
147
148         for (String tetangga_2014 :
149             graph_2014.get(current_2014)) {
150             if (!visited_2014.contains(tetangga_2014)) {
151                 visited_2014.add(tetangga_2014);
152                 parent_2014.put(tetangga_2014, current_2014);
153                 queue_2014.offer(tetangga_2014);
154             }
155         }
156     }
157
158     tampilkanHasil_2014(parent_2014, start_2014, goal_2014,
159     "BFS");
160 }
161
162 private void DFS_2014() {
163     resetData_2014();
164
165     String start_2014 =
166     cmbStart_2014.getSelectedItem().toString();

```

```

162     String goal_2014 =
    cmbGoal_2014.getSelectedItem().toString();
163
164     Stack<String> stack_2014 = new Stack<>();
165     HashSet<String> visited_2014 = new HashSet<>();
166     HashMap<String, String> parent_2014 = new HashMap<>();
167
168     stack_2014.push(start_2014);
169
170     while (!stack_2014.isEmpty()) {
171         String current_2014 = stack_2014.pop();
172
173         if (!visited_2014.contains(current_2014)) {
174             visited_2014.add(current_2014);
175             kunjungan_2014.add(current_2014);
176
177             if (current_2014.equals(goal_2014))
178                 break;
179
180             for (String tetangga_2014 :
graph_2014.get(current_2014)) {
181                 if (!visited_2014.contains(tetangga_2014)) {
182                     parent_2014.put(tetangga_2014,
current_2014);
183                     stack_2014.push(tetangga_2014);
184                 }
185             }
186         }
187     }
188
189     tampilkanHasil_2014(parent_2014, start_2014, goal_2014,
"DFS");
190 }
191
192 private void tampilkanHasil_2014(HashMap<String, String>
parent_2014,
193                                 String start_2014,
194                                 String goal_2014,
195                                 String metode_2014) {
196
197     jalur_2014.clear();
198
199     String current_2014 = goal_2014;
200
201     while (current_2014 != null) {
202         jalur_2014.add(0, current_2014);
203         current_2014 = parent_2014.get(current_2014);
204     }
205
206     hasil_2014.setText(
207         "Metode : " + metode_2014 +
208         "\nJalur : " + jalur_2014 +
209         "\nUrutan Kunjungan : " + kunjungan_2014 +
210         "\nJumlah Node Dieksplorasi : " +
kunjungan_2014.size()
211     );
212

```

```

213     panelGraph_2014.repaint();
214 }
215
216 private void displayGraph_2014(Graphics g) {
217     for (String node_2014 : graph_2014.keySet()) {
218         Point p1_2014 = posisi_2014.get(node_2014);
219
220         for (String tetangga_2014 : graph_2014.get(node_2014))
221         {
222             Point p2_2014 = posisi_2014.get(tetangga_2014);
223
224             g.setColor(Color.BLACK);
225             g.drawLine(p1_2014.x, p1_2014.y,
226                       p2_2014.x, p2_2014.y);
227         }
228     }
229
230     for (String node_2014 : posisi_2014.keySet()) {
231         Point p_2014 = posisi_2014.get(node_2014);
232
233         if (jalur_2014.contains(node_2014))
234             g.setColor(Color.GREEN);
235         else if (kunjungan_2014.contains(node_2014))
236             g.setColor(Color.YELLOW);
237         else
238             g.setColor(Color.LIGHT_GRAY);
239
240         g.fillOval(p_2014.x - 20,
241                  p_2014.y - 20,
242                  40,
243                  40);
244
245         g.setColor(Color.BLACK);
246         g.drawOval(p_2014.x - 20,
247                  p_2014.y - 20,
248                  40,
249                  40);
250
251         g.drawString(node_2014,
252                     p_2014.x - 25,
253                     p_2014.y - 25);
254     }
255 }
256
257 private void resetGraph_2014() {
258     resetData_2014();
259     hasil_2014.setText("");
260     panelGraph_2014.repaint();
261 }
262
263 private void resetData_2014() {
264     jalur_2014.clear();
265     kunjungan_2014.clear();
266 }
267
268 public static void main(String[] args) {

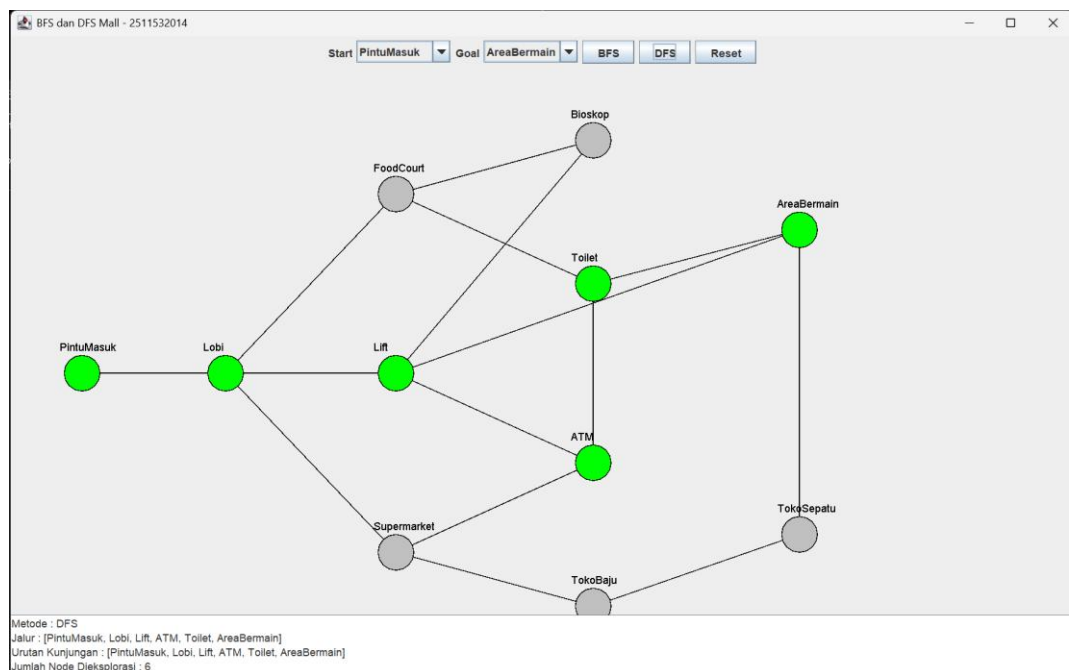
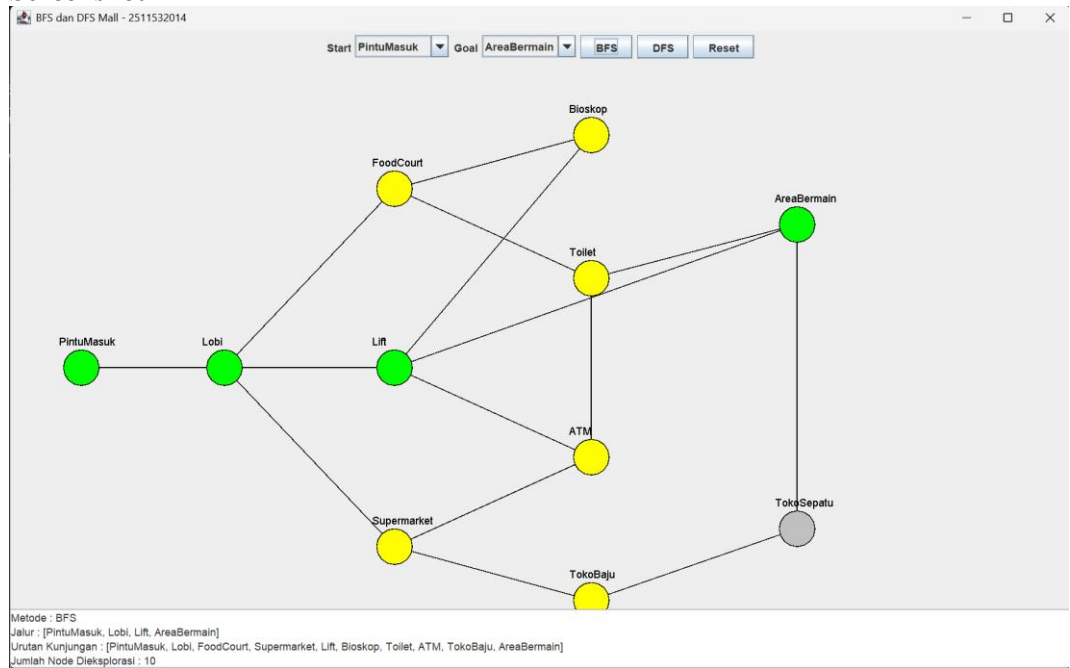
```

```

269         SwingUtilities.invokeLater(() -> {
270             new PetaMall_2511532014().setVisible(true);
271         });
272     }
273 }
274

```

1.1.2 Screenshot



1.1.3 Laporan singkat

1.1.3.1 Daftar Node

1. PintuMasuk
2. Lobi

3. FoodCourt
4. Bioskop
5. Supermarket
6. TokoBaju
7. TokoSepatu
8. ATM
9. Toilet
10. Lift
11. AreaBermain

1.1.3.2 Daftar Edge

1. PintuMasuk - Lobi
2. Lobi - FoodCourt
3. Lobi - Supermarket
4. Lobi - Lift
5. FoodCourt - Bioskop
6. FoodCourt - Toilet
7. Supermarket - ATM
8. Supermarket - TokoBaju
9. TokoBaju - TokoSepatu
10. TokoSepatu - AreaBermain
11. Lift - Bioskop
12. Lift - AreaBermain
13. ATM - Toilet
14. ATM - Lift
15. Toilet - AreaBermain

1.1.3.3 Hasil BFS

Titik Awal : Pintu Masuk

Titik Akhir : AreaBermain

Hasil Penelusuran:

- Urutan Kunjungan (Eksplorasi): [PintuMasuk, Lobi, FoodCourt, Supermarket, Lift, Bioskop, Toilet, ATM, TokoBaju, AreaBermain]
- Jalur Terpendek yang Ditemukan: [PintuMasuk, Lobi, Lift, AreaBermain]
- Jumlah Node Dieksplorasi: 10 Node

Analisis: BFS bergerak secara melebar. Dari Lobi, ia memeriksa seluruh tetangga terdekatnya dulu (FoodCourt, Supermarket, Lift) sebelum melangkah lebih dalam, sehingga dijamin menemukan rute dengan jumlah lompatan (edge) paling sedikit.

1.1.3.4 Hasil DFS

Titik Awal: Pintu Masuk

Titik Tujuan : AreaBermain

Hasil Penelusuran:

- Urutan Kunjungan (Eksplorasi): [PintuMasuk, Lobi, Lift, AreaBermain]
- Jalur yang Ditemukan: [PintuMasuk, Lobi, Lift, AreaBermain]
- Jumlah Node Dieksplorasi: 4 Node

Analisis: DFS bergerak secara mendalam. Kebetulan berdasarkan urutan push ke dalam Stack pada struktur graf ini, DFS langsung memilih cabang Lift dari Lobi, lalu dari Lift langsung memilih cabang menuju AreaBermain (Goal). Sehingga target langsung ditemukan dengan sangat cepat.

1.1.3.5 Kesimpulan perbandingan

Karakteristik	BFS (Breadth-First Search)	DFS (Depth-First Search)
Optimalitas Jalur	Sangat Optimal. Selalu menjamin pencarian rute terpendek (jumlah <i>edge</i> paling minimal).	Belum tentu optimal. Jalur yang ditemukan bergantung pada urutan tumpukan (<i>stack</i>).
Kecepatan Menemukan Target	Cenderung lebih lambat karena harus memeriksa semua node di level yang sama terlebih dahulu.	Bisa sangat cepat jika target berada di jalur dalam yang dipilih pertama kali (seperti kasus di atas).
Jumlah Eksplorasi Node	Lebih banyak (10 Node), karena memeriksa area mall secara menyeluruh per radius jarak.	Lebih sedikit (4 Node) pada kasus ini, karena langsung menelusuri ke dalam rute yang tepat.
Penggunaan Memori	Lebih besar karena menyimpan semua node bertetangga ke dalam <i>Queue</i> .	Lebih efisien karena hanya menyimpan node sepanjang jalur penelusuran aktif di dalam <i>Stack</i> .